

OpenClaw Natural Interaction + SLAM Navigation

OpenClaw Natural Interaction + SLAM Navigation

1. Course Content
 - Learning Objectives
2. Preparation
3. Record the Map Mapping File
4. Case Demonstration: OpenClaw with Regular Navigation
5. Case Demonstration: OpenClaw with Road Network Planning Navigation
6. Source Code Analysis
 - 6.1 Navigation — Navigation Function
 - 6.2 PreciseAdjustPosition — Precise Position Adjustment
7. Parameter Debugging

1. Course Content

- Use OpenClaw to control the robot to perform semantic navigation in the map combined with map mapping, so that the robot can go to a specified location to perform tasks

Learning Objectives

Basic

- Learn to use the RecordMapLocation tool to record the map mapping file and establish the correspondence between place names and coordinates
- Learn to use OpenClaw to issue a task to the robot to go to the target point

Advanced

- Understand how to switch between Road Network Navigation and regular navigation (Nav2)
- Master the PID control principle and parameter tuning methods of PreciseAdjustPosition for precise position adjustment

2. Preparation

[!IMPORTANT]

- OpenClaw combined with the regular navigation feature requires building a grid map and a pose map first. Refer to the tutorial on LiDAR Gameplay / slam_toolx Mapping.
- OpenClaw combined with the road network navigation feature requires building a grid map, a pose map, and annotating the road network file first. Refer to the tutorial on Road Network Planning Navigation — Building Pose Maps, Road Network Annotation.

```
ros2 launch microros_v2_bringup openclaw_service.launch.py use_nav:=true
```

- Use the 2D Pose Estimate tool to initialize the robot's position in the map



3. Record the Map Mapping File

[!TIP]

- You can use the robot_control CLI tool, or directly ask OpenClaw to record positions and symbols during interaction with OpenClaw. The following demonstrates recording a map mapping file using the CLI tool as an example

Tool	Parameters	Function
RecordMapLocation	<code>--name</code> , <code>--symbol</code>	Save the robot's current position in the map to the map mapping file

- You can use the robot_control CLI tool to record the position of the robot chassis **base_link** in the map **map**

```
robot_control call-tool RecordMapLocation --name 发货区 --symbol B
```

- `--name` specifies the name of the recorded point
- `--symbol` specifies the symbol of the recorded point, so that OpenClaw can identify and distinguish it

```
yahboom@yahboom:~/map$ robot_control call-tool RecordMapLocation --name 发货区 --symbol B
{
  "execution_result": "success",
  "reason": ""
}
```

- If the recording succeeds, the content shown in the figure above will be prompted. If a symbol with the same name already exists in the map mapping file, the previous symbol content will be automatically overwritten

- map_mapping.yaml, the file path is:

```
$HOME/microros_ws/src/microros_v2_bringup/config/map_mapping.yaml
```

```
yahboom@yahboom:~/map$ cat $HOME/microros_ws/src/microros_v2_bringup/config/map_mapping.yaml
A:
  name: 茶水间
  position:
    x: 3.975
    y: 0.699
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.495
    w: 0.869
B:
  name: 发货区
  position:
    x: 2.08
    y: 0.325
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.162
    w: 0.987
```

4. Case Demonstration: OpenClaw with Regular Navigation

[!TIP]

- You can write the commands according to your own needs. The following is a case demonstration. You can choose any interaction method; here we demonstrate with Web Chat on the web page as an example
- Navigation function tools

Tool	Parameters	Function
GetMapMapping	None	Get the mapping relationship between all place names and their corresponding identifiers
Navigation	--location	Navigate to the target position via the target point identifier

Navigate to Shipping Zone

openclaw-control-ui 20:34

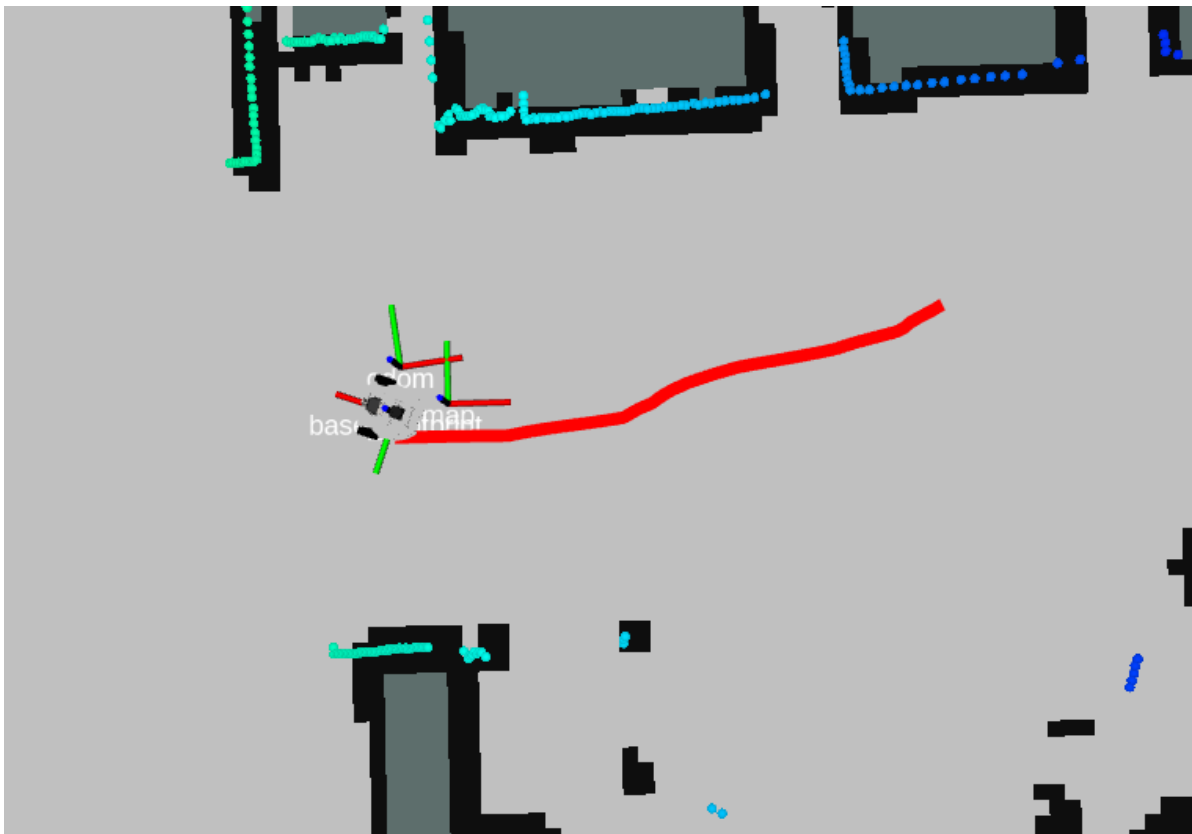
» 🔧 1 tool read

» 🔧 1 tool GetMapMapping

Locate 'Shipping Zone' on the map, corresponding to symbol M, and start navigation.

» 🔧 1 tool Navigation

Successfully navigated to Shipping Zone 🚚



5. Case Demonstration: OpenClaw with Road Network Planning Navigation

```
ros2 launch microros_v2_bringup openclaw_service.launch.py use_nav:=true
nav_mode:=roadnet
```

- Here we demonstrate navigating to the Shipping Area as an example

Navigate to Shipping Zone
openclaw-control-ui 20:34

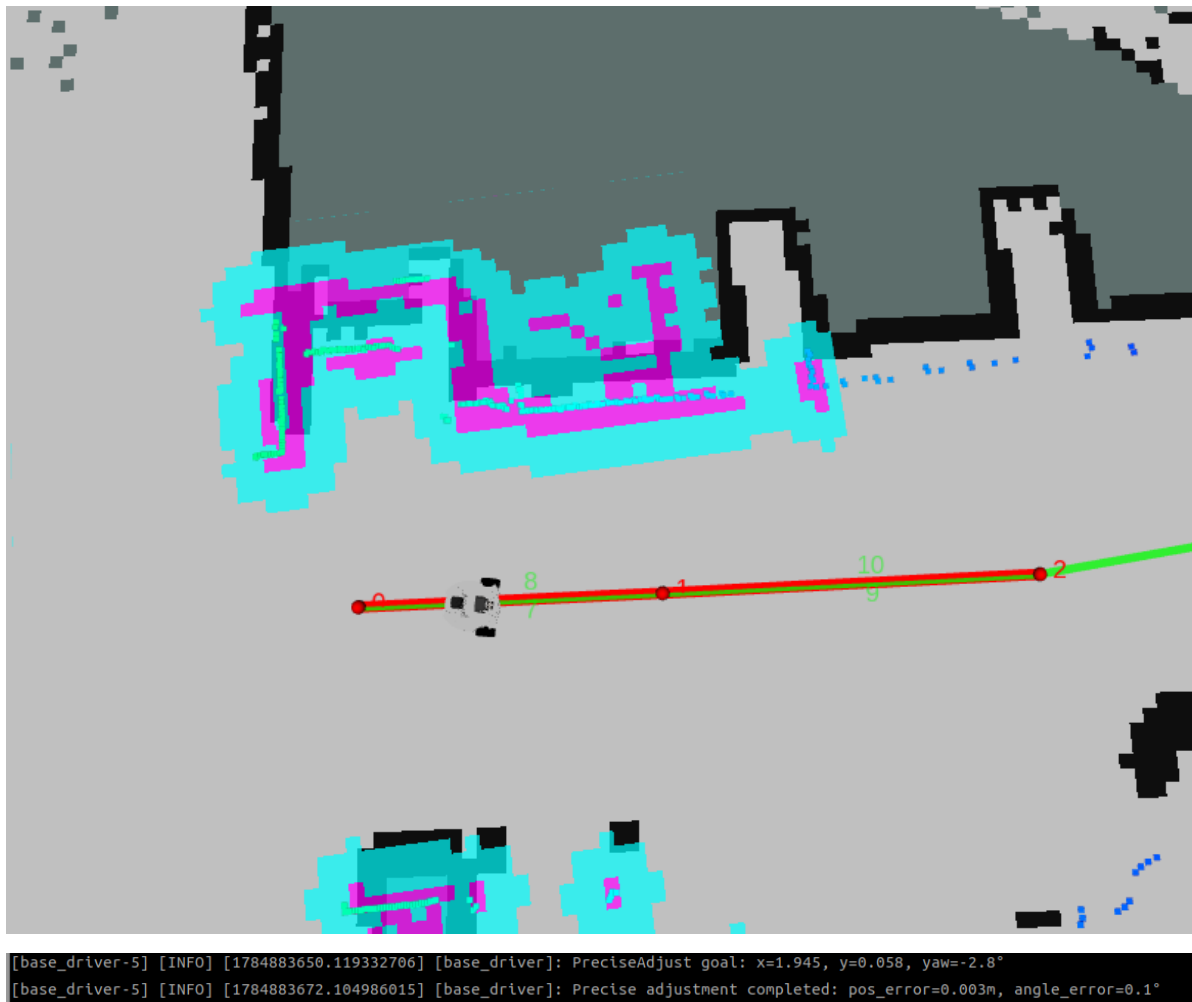
▶ 🔧 1 tool read

▶ 🔧 1 tool GetMapMapping

Locate 'Shipping Zone' on the map, corresponding to symbol **M**, and start navigation.
 ▶ 🔧 1 tool Navigation

Successfully navigated to Shipping Zone 🏠

- The robot will navigate along the path in the road network to the target point and adjust its pose



6. Source Code Analysis

- Source Code Path

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

6.1 Navigation — Navigation Function

`Navigation()` is the navigation entry function in `action_controller.py`. It determines the navigation mode based on the startup parameters `use_nav` and `nav_mode`. The overall flow is divided into three steps: read the map mapping file → construct the target pose → dispatch to the specific navigation implementation.

1. **Read the map mapping file:** Opens `map_mapping.yaml`, iterates over all symbol entries, matches the `location` parameter (supports symbol matching or name matching), and finds the target coordinate data
2. **Construct the target pose:** Fills the matched position (x, y, z) and orientation (quaternion) into the `PoseStamped` message, with the coordinate frame set to `map`
3. **Navigation mode dispatch:**
 - `nav_mode == 'nav2'`: calls `_nav2_navigation()` to perform regular navigation through Nav2's `NavigateToPose` Action service
 - `nav_mode == 'roadnet'`: calls `_roadnet_navigation()` to perform road network navigation through the `RoadNetToGoal` Action service

4. **Precise position adjustment:** Regardless of the navigation mode, after reaching the target, `PreciseAdjustPosition()` is called to fine-tune the pose and improve the final docking accuracy

`_nav2_navigation()`: Constructs `NavigateToPose.Goal()` and sends it through `nav_to_pose_client`, waits asynchronously for the execution result, and calls the precise position adjustment upon completion

`_roadnet_navigation()`: Constructs `RoadNetToGoal.Goal()`, sets `use_poses=True` to use coordinates directly, sends it through `roadnet_nav_client`, and determines the navigation result based on `result.success`

```
def Navigation(self, location: str) -> list[bool, str]:
    '''Navigate to target location. Mode depends on use_nav and nav_mode
    parameters.'''
    if not self.use_nav:
        return [False, "Navigation is not enabled ,Please add the startup
        parameter `use_nav:=true` when starting the OpenClaw service.This step should
        only be performed by the user."]

    location = location.strip("\\"")
    try:
        with open(self.map_mapping_file, 'r', encoding='utf-8') as file:
            map_data = yaml.safe_load(file) or {}

        target_data = None
        for symbol, data in map_data.items():
            if symbol == location or (isinstance(data, dict) and
            data.get('name') == location):
                target_data = data
                break
        if target_data is None:
            error_info = f"The map mapping file has no coordinate record for
            '{location}'."
            self.get_logger().error(error_info)
            return [False, error_info]
    except Exception as e:
        self.get_logger().error(str(e))
        return [False, str(e)]

    goal_msg = PoseStamped()
    goal_msg.header.frame_id = "map"
    goal_msg.header.stamp = self.get_clock().now().to_msg()
    position = target_data.get('position', {})
    goal_msg.pose.position.x = float(position.get('x', 0.0))
    goal_msg.pose.position.y = float(position.get('y', 0.0))
    goal_msg.pose.position.z = float(position.get('z', 0.0))
    orientation = target_data.get('orientation', {})
    goal_msg.pose.orientation.x = float(orientation.get('x', 0.0))
    goal_msg.pose.orientation.y = float(orientation.get('y', 0.0))
    goal_msg.pose.orientation.z = float(orientation.get('z', 0.0))
    goal_msg.pose.orientation.w = float(orientation.get('w', 1.0))

    if self.nav_mode == 'nav2':
        return self._nav2_navigation(goal_msg, position, orientation,
        location)
    elif self.nav_mode == 'roadnet':
```

```

        return self._roadnet_navigation(goal_msg, location)
    else:
        error=f"Unknown nav_mode: '{self.nav_mode}'. Please check the
nav_mode startup parameters."
        self.get_logger().error(error)
        return [False, error]

    def _nav2_navigation(self, goal_msg: PoseStamped, position: dict,
orientation: dict, location: str) -> list[bool,str]:
        '''Navigate using nav2 NavigateToPose action'''
        if not self.nav_to_pose_client.wait_for_server(timeout_sec=10.0):
            return [False, "Nav2 action server not available,Users should check
if the navigation nodes are functioning correctly"]
        goal = NavigateToPose.Goal()
        goal.pose.header.frame_id = "map"
        goal.pose.header.stamp = self.get_clock().now().to_msg()
        goal.pose.pose.position.x = float(position.get('x', 0.0))
        goal.pose.pose.position.y = float(position.get('y', 0.0))
        goal.pose.pose.position.z = float(position.get('z', 0.0))
        goal.pose.pose.orientation.x = float(orientation.get('x', 0.0))
        goal.pose.pose.orientation.y = float(orientation.get('y', 0.0))
        goal.pose.pose.orientation.z = float(orientation.get('z', 0.0))
        goal.pose.pose.orientation.w = float(orientation.get('w', 1.0))

        send_goal_future = self.nav_to_pose_client.send_goal_async(goal)
        while rclpy.ok() and not send_goal_future.done():
            time.sleep(0.5)

        if not send_goal_future.done():
            return [False, "Failed to send navigation goal"]

        goal_handle = send_goal_future.result()
        if not goal_handle.accepted:
            return [False, "Navigation goal rejected"]

        result_future = goal_handle.get_result_async()

        while rclpy.ok() and not result_future.done():
            time.sleep(0.5)

        if result_future.done():
            result = result_future.result().result
            self.PreciseAdjustPosition(goal_msg.pose)
            return [True, "Navigation succeeded"]
        else:
            return [False, "Navigation failed"]

    def _roadnet_navigation(self, goal_msg: PoseStamped, location: str) ->
list[bool,str]:
        '''Navigate using roadnet action'''
        if not self.roadnet_nav_client.wait_for_server(timeout_sec=10.0):
            return [False, "RoadNet action server not available,Users should
check if the navigation nodes are functioning correctly."]

        goal = RoadNetToGoal.Goal()
        goal.use_start = False
        goal.use_poses = True
        goal.goal = goal_msg

```

```

        self.get_logger().info(f"Sending roadnet navigation goal to
'{location}'")

        send_goal_future = self.roadnet_nav_client.send_goal_async(goal)
        while rclpy.ok() and not send_goal_future.done():
            time.sleep(0.5)

        if not send_goal_future.done():
            return [False, "Failed to send roadnet navigation goal"]

        goal_handle = send_goal_future.result()
        if not goal_handle.accepted:
            return [False, "Roadnet navigation goal rejected"]

        result_future = goal_handle.get_result_async()

        while rclpy.ok() and not result_future.done():
            time.sleep(0.5)

        if result_future.done():
            result = result_future.result().result
            if result.success:
                self.PreciseAdjustPosition(goal_msg.pose)
                return [True, f"Roadnet navigation to '{location}' succeeded"]
            else:
                self.get_logger().warn(
                    Fore.YELLOW + f"Roadnet navigation to '{location}' failed:
                    f"error_code={result.error_code}, {result.error_msg}" + Fore.RESET)
                return [False, result.error_msg]
        else:
            return [False, "Roadnet navigation failed"]

    def PreciseAdjustPosition(self, target_pose: Pose) -> list[bool, str]:
        '''Request precise pose adjustment via PreciseAdjustPose action
        service'''
        if not self.precise_adjust_client.wait_for_server(timeout_sec=10.0):
            return [False, "PreciseAdjustPose action server not available"]

        goal = PreciseAdjustPose.Goal()
        goal.target_pose = target_pose

        send_goal_future = self.precise_adjust_client.send_goal_async(goal)
        while rclpy.ok() and not send_goal_future.done():
            time.sleep(0.5)

        if not send_goal_future.done():
            return [False, "Failed to send PreciseAdjustPose goal"]

        goal_handle = send_goal_future.result()
        result_future = goal_handle.get_result_async()
        while rclpy.ok() and not result_future.done():
            time.sleep(0.5)

        if not result_future.done():
            return [False, "PreciseAdjustPose result timeout"]

        result = result_future.result().result

```



```

if result.success:
    return [True, result.message]
return [False, result.message]

```

6.2 PreciseAdjustPosition — Precise Position Adjustment

`_preciseAdjustActionCallback()` is the PID-based precise pose adjustment implementation in `base_driver.py`. It runs as the callback function of the `PreciseAdjustPose` Action server. It adopts a three-stage state machine strategy and gradually converges to the target pose through visual/TF localization feedback

Three-stage state machine:

1. **AIM (aiming stage):** Rotates in place to point the front of the vehicle toward the target point; when `bearing_error < aim_tol` (about 5.7°), it transitions to the DRIVE stage
2. **DRIVE (forward stage):** Drives straight toward the target point while correcting the heading deviation with angular velocity; if `bearing_error > realign_tol` (about 20°) during driving, it returns to the AIM stage to re-aim
3. **FINAL_YAW (final heading stage):** After reaching the target position, rotates in place to adjust to the target yaw angle; if the position deviation exceeds `reenter_dist`, it returns to the AIM stage and re-enters

Control logic:

1. **Error calculation:** Obtains the current pose in the map coordinate frame through `_getMapPose()`, and calculates the position error `pos_error`, heading error `yaw_error`, and bearing error `bearing_error`
2. **P control:** Uses pure proportional control. `xy_kp` maps the distance error to forward speed, and `theta_kp` maps the angle error to angular speed, both limited within the maximum/minimum speed ranges
3. **Completion check:** Only when both the position error and the angle error are less than the tolerance and the condition is satisfied 5 consecutive times (`required_success=5`) is it considered successful, avoiding misjudgment from a single jitter

```

def _preciseAdjustActionCallback(self, goal_handle):

    goal = goal_handle.request
    target_x = goal.target_pose.position.x
    target_y = goal.target_pose.position.y
    target_yaw = self._getYawFromQuaternion(goal.target_pose.orientation)

    self.get_logger().info(
        f"PreciseAdjust goal: x={target_x:.3f}, y={target_y:.3f}, yaw=
{math.degrees(target_yaw):.1f}°")

    # Gains
    xy_kp = self.precise_adjust_xy_pid[0]          # distance -> forward speed
    theta_kp = self.precise_adjust_theta_pid[0]    # heading error -> angular
speed

    consecutive_success = 0
    required_success = 5

```

```

        # Heading thresholds (rad): must face the target within aim_tol before
        driving;
        # while driving, fall back to AIM if the bearing drifts beyond
        realign_tol.
        aim_tol = 0.10          # ~5.7 deg
        realign_tol = 0.35      # ~20 deg
        # Leave FINAL_YAW back to AIM only after clearly drifting out of
        tolerance.
        reenter_dist = self.precise_adjust_pos_tolerance * 1.5

        phase = self._PHASE_AIM
        pos_error = 0.0
        yaw_error = 0.0

        for _ in range(self.precise_adjust_max_iteration):
            # Cancel check
            if goal_handle.is_cancel_requested:
                self._publishCmdVel(0.0, 0.0)
                goal_handle.canceled()
                result = PreciseAdjustPose.Result()
                result.success = False
                result.message = "Precise adjustment cancelled"
                return result

            pose = self._getMapPose()
            if pose is None:
                self.get_logger().error("Failed to get current pose from TF")
                self._publishCmdVel(0.0, 0.0)
                goal_handle.abort()
                result = PreciseAdjustPose.Result()
                result.success = False
                result.message = "Failed to get current pose from TF"
                return result

            current_x, current_y, current_yaw = pose

            # Position/orientation error in the map frame
            dx = target_x - current_x
            dy = target_y - current_y
            pos_error = math.hypot(dx, dy)
            yaw_error = self._normalizeAngle(target_yaw - current_yaw)
            bearing_error = self._normalizeAngle(math.atan2(dy, dx) -
            current_yaw)

            pos_reached = pos_error < self.precise_adjust_pos_tolerance
            angle_reached = abs(yaw_error) < self.precise_adjust_angle_tolerance

            # Publish feedback
            feedback = PreciseAdjustPose.Feedback()
            feedback.position_error = float(pos_error)
            feedback.angle_error = float(math.degrees(yaw_error))
            goal_handle.publish_feedback(feedback)

            if pos_reached and angle_reached:
                consecutive_success += 1
                if consecutive_success >= required_success:
                    self._publishCmdVel(0.0, 0.0)

```

```

        msg = (f"Precise adjustment completed: pos_error={
pos_error:.3f}m, "
                f"angle_error={math.degrees(yaw_error):.1f}°")
        self.get_logger().info(msg)
        goal_handle.succeed()
        result = PreciseAdjustPose.Result()
        result.success = True
        result.message = msg
        return result
    else:
        consecutive_success = 0

# ---- Phase transitions (with hysteresis) ----
    if pos_reached:
        phase = self._PHASE_FINAL_YAW
    elif phase == self._PHASE_FINAL_YAW:
        # was aligning yaw but drifted off the point: re-aim and drive
back in.

        if pos_error > reenter_dist:
            phase = self._PHASE_AIM
        elif phase == self._PHASE_AIM:
            if abs(bearing_error) < aim_tol:
                phase = self._PHASE_DRIVE
        elif phase == self._PHASE_DRIVE:
            if abs(bearing_error) > realign_tol:
                phase = self._PHASE_AIM

# ---- Per-phase velocity command ----
    if phase == self._PHASE_DRIVE:
        # Drive forward toward the target; heading correction keeps it on
bearing.

        vx = xy_kp * pos_error * max(0.0, math.cos(bearing_error))
        vx = max(0.0, min(self.precise_adjust_max_linear_vel, vx))
        if vx > 1e-6 and vx < self.min_linear_speed:
            vx = self.min_linear_speed
        omega = theta_kp * bearing_error
        omega = max(-self.precise_adjust_max_angular_vel,
                    min(self.precise_adjust_max_angular_vel, omega))
    else:
        # In-place rotation: AIM faces the target point, FINAL_YAW the
target yaw.

        if phase == self._PHASE_FINAL_YAW:
            heading_error = yaw_error
            reached = angle_reached
        else:
            heading_error = bearing_error
            reached = abs(bearing_error) < aim_tol
        vx = 0.0
        omega = theta_kp * heading_error
        omega = max(-self.precise_adjust_max_angular_vel,
                    min(self.precise_adjust_max_angular_vel, omega))
        if reached:
            omega = 0.0
        elif abs(omega) < self.min_angular_speed:
            # Overcome the chassis static-friction dead-zone so it
actually turns.

            omega = self.min_angular_speed if omega > 0 else -
self.min_angular_speed

```

```

        # Publish command (linear.y = 0 for differential drive)
        self._publishCmdVel(vx, omega)
        time.sleep(0.1) # 10Hz

    # Max iterations reached
    self._publishCmdVel(0.0, 0.0)
    msg = (f"Precise adjustment reached max iterations
({self.precise_adjust_max_iteration}). "
          f"pos_error={pos_error:.3f}m, angle_error=
{math.degrees(yaw_error):.1f}°")
    self.get_logger().warn(msg)
    goal_handle.abort()
    result = PreciseAdjustPose.Result()
    result.success = False
    result.message = msg
    return result

```

7. Parameter Debugging

- Parameter file path:

```
$HOME/microros_ws/src/microros_v2_bringup/config/common.yaml
```

The following describes the parameters related to precise position adjustment in `common.yaml`:

Parameter	Default	Description
<code>precise_adjust_xy_pid</code>	[0.5, 0.0, 0.05]	PID gains for the DRIVE stage [Kp, Ki, Kd]; Kp maps the distance error to forward speed
<code>precise_adjust_theta_pid</code>	[1.0, 0.0, 0.3]	PID gains for the turning stage [Kp, Ki, Kd]; Kp maps the heading error to angular speed
<code>precise_adjust_max_linear_vel</code>	0.12	Maximum linear speed for precise adjustment (m/s); slow down to improve convergence stability
<code>precise_adjust_max_angular_vel</code>	0.5	Maximum angular speed for precise adjustment (rad/s); ensures decisive turning
<code>precise_adjust_pos_tolerance</code>	0.01	Position tolerance for precise adjustment (m); loosen to accommodate TF/odometry noise
<code>precise_adjust_angle_tolerance</code>	0.06	Angle tolerance for precise adjustment (rad, about 3.4°)
<code>precise_adjust_max_iteration</code>	500	Maximum iteration count for precise adjustment (10Hz → about 50s); aborts on timeout

[!TIP]

- If the robot still has obvious offset after reaching the target position, increase the Kp value of `precise_adjust_xy_pid` or loosen `precise_adjust_pos_tolerance`
- If the robot's final heading is not accurate, increase the Kp value of `precise_adjust_theta_pid` or loosen `precise_adjust_angle_tolerance`
- If the robot jitters/oscillates during adjustment, reduce the PID gains or decrease `precise_adjust_max_linear_vel`
- If the precise adjustment takes too long, increase `precise_adjust_max_iteration` or appropriately loosen the tolerances

```

base_driver:
  ros__parameters:
    ....
    precise_adjust_xy_pid: [0.5, 0.0, 0.05]      # DRIVE stage: Kp = distance -
> forward speed gain [Kp, Ki, Kd]
    precise_adjust_theta_pid: [1.0, 0.0, 0.3]    # Each turning stage: Kp =
heading error -> angular speed gain [Kp, Ki, Kd]
    precise_adjust_max_linear_vel: 0.12         # Max linear speed for precise
adjustment (m/s), slow down to improve convergence stability
    precise_adjust_max_angular_vel: 0.5        # Max angular speed for
precise adjustment (rad/s), ensures decisive turning
    precise_adjust_pos_tolerance: 0.01         # Precise adjustment position
tolerance (m), loosen to accommodate TF/odometry noise
    precise_adjust_angle_tolerance: 0.06       # Precise adjustment angle
tolerance (rad, ~3.4°)
    precise_adjust_max_iteration: 500         # Precise adjustment max
iterations (10Hz → ~50s)
    ...

```